Sayly

Product Requirements Document

Enhanced MVP / Phase 1 Product Specification

Mobile-first language exchange and social learning application
Frontend: React Native + TypeScript | Platforms: Android & iOS

Core product loop

Create profile → Set languages → Discover people → Connect → Chat → Practice regularly

Executive Summary

Sayly is a mobile-first language exchange product designed around one high-value loop: help a learner find a relevant language partner and move from discovery to a real conversation with as little friction as possible. Phase 1 should prioritize profile quality, language-aware discovery, a controlled connection system, one-to-one messaging, and basic safety.

> **Product principle:** build the conversation-starting experience exceptionally well before adding AI tutors, calls, subscriptions, communities, gamification, or other advanced features.

Phase 1 success path

Install → Register → Complete profile → Add spoken/learning languages → Discover a relevant partner → View profile → Send request → Request accepted → Start chat → Exchange real-time messages

Product Requirements Document (PRD)

1. Product Overview

Product Name

Sayly

Product Type

Mobile-first language exchange and social learning application.

Platform

• **Frontend:** React Native + TypeScript

• **Backend:** Node.js + TypeScript

• **Primary platforms:** Android and iOS

Product Vision

Build a platform where language learners can **connect with native speakers and other learners to practice languages through real conversations**.

The core idea is similar to a language exchange app: a user specifies:

• Languages they speak

• Their proficiency in those languages

• Languages they want to learn

• Their current proficiency and goals

The platform then helps them discover and connect with suitable language partners. Once connected, users can communicate through messaging and practice their target languages together.

The Phase 1 product should focus on one core loop:

> **Create profile → Set languages → Discover people → Connect → Chat → Practice regularly**

The goal of Phase 1 is to build this loop well before adding more complex features such as video calls, AI tutors, subscriptions, or advanced social features.

---

2. Problem Statement

Learning a language requires regular exposure and real communication. Many learners face problems such as:

• Not having access to native speakers

• Difficulty finding reliable language partners

• Not knowing whether another person's language level matches their needs

• Using multiple apps to find people and communicate

• Losing language partners because there is no structured connection system

• Feeling uncomfortable starting conversations with strangers

Existing language exchange platforms can also become overwhelming because of:

• Too many irrelevant users

• Poor matching

• Spam

• Incomplete profiles

• Difficulty identifying serious learners

This app should make finding a suitable practice partner **simple, relevant, and safe**.

---

3. Target Users

3.1 Primary User: Language Learner

A person learning one or more languages who wants to practice with real people.

Example

Name: Aryan

Speaks: Hindi — Native

Speaks: English — Advanced

Learning: Russian — Beginner

The user wants to find someone who:

• Speaks Russian fluently or natively

• Is interested in learning English or Hindi

• Wants regular conversations

3.2 Native Speaker / Language Partner

A person who speaks a language fluently and wants to:

• Help others practice

• Learn another language in exchange

• Meet people from different countries

• Practice through conversation

3.3 Secondary User

A multilingual person who wants to:

• Make international friends

• Maintain a language

• Participate in language communities

• Practice several languages

---

4. Phase 1 Goals

The first version must allow users to:

1. Create an account
2. Create a meaningful language profile
3. Add languages they speak
4. Add languages they are learning
5. Set proficiency levels
6. Discover relevant users
7. Search and filter potential language partners
8. View other user profiles
9. Send a connection request
10. Accept or decline connection requests
11. Chat with connected users
12. Receive essential notifications
13. Block or report problematic users
14. Manage their profile and settings

---

5. Phase 1 Non-Goals

The following features should **not be required for the initial launch**:

• Video calling

• Voice calling

• AI language tutor

• AI conversation generation

• Paid subscriptions

• Advanced gamification

• Public social feed

• Stories

• Live streaming

• Large group communities

• Lessons or structured language courses

• Marketplace for professional tutors

• Complex matching algorithms

• Desktop/web application

The architecture should support these features later, but they should not increase the complexity of Phase 1 unnecessarily.

---

6. Core User Journey

6.1 New User Journey

```
Open App
↓
Welcome / Introduction
↓
Sign Up or Log In
↓
Verify Account
↓
Create Basic Profile
↓
Add Languages I Speak
↓
Add Languages I Want to Learn
↓
Select Proficiency Levels
↓
Select Practice Preferences
↓
Discover Language Partners
↓
View Profile
↓
Send Connection Request
↓
Request Accepted
↓
Start Conversation
↓
Continue Practicing
```

---

7. Functional Requirements

7.1 Authentication

Users must be able to securely create and access their accounts.

Required Features

Sign Up

Support:

• Email and password

• Google authentication

Other providers such as Apple can be added later.

Login

Users can log in using their registered authentication method.

Logout

Users can securely log out.

Password Reset

Users should be able to:

1. Enter their email

2. Request a password reset

3. Receive a reset method/link

4. Set a new password

Authentication Session

The application must:

• Persist the user's session securely

• Restore the session when the app is reopened

• Refresh authentication credentials when appropriate

• Redirect unauthenticated users to authentication screens

7.2 User Onboarding

After registration, users should complete onboarding before accessing the main application.

Step 1: Basic Information

Required:

• Display name

• Date of birth or age confirmation

• Gender — optional depending on product policy

• Country

• Profile photo

Optional:

• Short bio

Bio Example

> "Learning Russian and looking for people to practice with. I can help with English and Hindi."

Step 2: Languages I Speak

The user adds one or more languages.

For each language:

• Language name

• Proficiency level

Recommended levels:

• Native

• Fluent

• Advanced

• Intermediate

• Beginner

At least one spoken language should be required.

Step 3: Languages I Want to Learn

For each target language:

• Language name

• Current proficiency

Recommended levels:

• Complete Beginner

• Beginner

• Elementary

• Intermediate

• Upper Intermediate

• Advanced

At least one target language should be required for Phase 1.

Step 4: Practice Preferences

Allow users to specify:

• Languages they want to practice

• Preferred partner language(s)

• Practice goals

Practice goals can include:

• Conversation

• Speaking

• Grammar

• Vocabulary

• Pronunciation

• Making friends

For Phase 1, this should remain simple.

Step 5: Onboarding Completion

After completing required information:

```
Profile Complete
↓
Enter Main Application
```

The backend should track whether onboarding has been completed.

7.3 User Profile

Every user should have a public profile visible to other users.

Profile Information

Display:

• Profile photo

• Display name

• Country

• Age range or age, depending on privacy policy

• Bio

• Languages spoken

• Proficiency levels

• Languages being learned

• Proficiency levels

• Practice goals

Example Profile

```
Aryan

India

About:
Learning Russian and looking for regular conversation practice.

Speaks:
■■ Hindi — Native
■■ English — Advanced

Learning:
■■ Russian — Beginner

Interested in:
Conversation
Vocabulary
Speaking
```

Edit Profile

Users can edit:

• Profile photo

• Display name

• Bio

• Country

• Spoken languages

• Learning languages

• Proficiency levels

• Practice goals

7.4 Language Management

The application requires a centralized language system.

The system should provide a predefined list of supported languages.

Each language should have:

• Unique ID

• Name

• Native name if needed

• ISO language code

• Display information

Example:

```
English
Code: en

Russian
Code: ru

Hindi
Code: hi
```

Users should select from supported languages instead of creating arbitrary language names.

This ensures:

• Consistent search

• Reliable matching

• Easier filtering

• Clean database relationships

7.5 User Discovery

User discovery is one of the most important Phase 1 features.

Users need a way to find suitable language partners.

Discovery Screen

The app should display recommended users.

Each user card should show:

• Profile photo

• Name

• Country

• Languages spoken

• Languages learning

• Proficiency

• Relevant language overlap

Example

```
Maria
Russia

Speaks:
Russian — Native

Learning:
English — Intermediate

Match:
You want to learn Russian
Maria wants to practice English
```

Discovery Priority

Initially, recommendations should prioritize users where there is a useful language relationship.

Priority Example

A strong match:

```
User A speaks Russian
User A learns English

User B speaks English
User B learns Russian
```

This is a mutual language exchange.

Other relevant matches may include:

```
Current user learns Russian
Other user speaks Russian
```

Even if the exchange is not perfectly mutual, the user can still be shown.

7.6 Search and Filters

Users should be able to find people manually.

Search

Phase 1 search should support:

• Display name

• Language

Filters

Essential filters:

• Language spoken

• Language being learned

• Country

• Proficiency level

Optional filters for later:

• Age range

• Distance

• Last active

• Availability

• Interests

Do not add advanced filters unless there is a clear product requirement.

7.7 Connection System

Users should not immediately have unrestricted access to private conversations.

Phase 1 should use a connection system.

Send Connection Request

A user can send a connection request.

Optionally, they can include a short message.

Example:

> "Hi! I'm learning Russian and can help you practice English. Would you like to practice together?"

Request States

A connection request can have the following states:

```
PENDING
ACCEPTED
DECLINED
CANCELLED
```

The backend must prevent duplicate active requests between the same users.

Accept Request

When accepted:

• A connection is created

• Both users become connected

• Messaging becomes available

• Both users receive a notification

Decline Request

The request is marked as declined.

The requester should not receive unnecessary details about why it was declined.

Cancel Request

The sender can cancel a pending request.

Remove Connection

Users can remove an existing connection.

After removal:

• The connection is deleted or deactivated

• Future messaging behavior follows the product's privacy rules

• The conversation should not automatically be deleted

7.8 Messaging

Messaging is the central interaction feature of Phase 1.

Conversation Types

For Phase 1:

```
One-to-One Conversation
```

Group chats should be added later only if needed.

Users Can

• Send text messages

• Receive messages in real time

• View conversation history

• See timestamps

• See delivery status

• See read status

• Open conversations from their chat list

Message Types for Phase 1

Required:

```
TEXT
```

Recommended if implementation is straightforward:

```
IMAGE
```

Voice messages, documents, reactions, and advanced media should be deferred.

Chat List

The chat screen should show:

• Profile image

• User name

• Last message

• Last activity/message time

• Unread message indicator

Real-Time Requirements

When User A sends a message:

```
User A sends message
↓
Backend stores message
↓
Real-time event is emitted
↓
User B receives message
↓
Conversation updates
```

The system should support users reconnecting after temporary network loss.

7.9 Message Status

Recommended Phase 1 statuses:

```
SENT
DELIVERED
READ
```

The UI can initially prioritize:

• Sent

• Read

The backend should be designed so additional delivery tracking can be improved later.

7.10 Notifications

Phase 1 notifications should focus only on important actions.

Required Notifications

Connection Request

> "Maria sent you a connection request."

Connection Accepted

> "Maria accepted your connection request."

New Message

> "Maria sent you a message."

Notification Types

The application should support:

• In-app notifications

• Push notifications

Notification preferences can initially remain simple.

7.11 Blocking

Users must be able to block another user.

When a user blocks someone:

• They should not receive messages from that user

• New connection requests should be prevented

• The blocked user should not appear in normal discovery

• The blocker should not appear in normal discovery for the blocked user where feasible

Blocking should work in both directions.

7.12 Reporting

Users should be able to report:

• A profile

• A message

Initial report categories:

• Spam

• Harassment

• Inappropriate content

• Fake profile

• Other

The report should contain:

• Reporter ID

• Reported user ID

• Optional related message ID

• Reason

• Additional details

• Timestamp

A basic administrative review process can initially be handled outside the user-facing mobile app.

7.13 Settings

The Phase 1 settings screen should include:

Account

• Edit profile

• Change password

• Log out

• Delete account

Privacy and Safety

• Blocked users

Notifications

• Enable/disable push notifications

Application

• App information

• Terms of service

• Privacy policy

7.14 Account Deletion

Users should be able to request account deletion.

The system should define whether data is:

• Immediately deleted, or

• Soft-deleted and permanently removed after a retention period

For Phase 1, a soft-delete strategy is recommended if legally and operationally appropriate.

Deleted users should:

• No longer appear in discovery

• Be unable to log in

• Have their active sessions invalidated

---

8. Main Screens — Phase 1

Authentication

```
Splash Screen
Welcome Screen
Sign Up
Log In
Forgot Password
Reset Password
```

Onboarding

```
Basic Information
Profile Photo
Languages I Speak
Languages I Learn
Proficiency Levels
Practice Preferences
```

Main Application

Recommended bottom navigation:

```
Discover | Connections | Chats | Profile
```

Discover

Find recommended language partners.

Connections

Manage:

• Pending requests

• Received requests

• Accepted connections

Chats

View conversations and messages.

Profile

View and edit the current user's profile.

Settings can be accessed through Profile.

---

9. Matching Logic — Phase 1

Phase 1 does not require machine learning.

Use a simple deterministic scoring approach.

Example Scoring Factors

Strong Positive Signal

The other user speaks a language that the current user is learning.

Additional Positive Signal

The current user speaks a language that the other user is learning.

Other Signals

• Shared practice goals

• Compatible proficiency levels

Example:

```
Current User:
Speaks English
Learning Russian

Other User:
Speaks Russian
Learning English
```

This receives a high match score.

The exact algorithm should remain internal and easy to modify.

---

10. Essential Data Entities

User

```
id
email
passwordHash
authProvider
status
onboardingCompleted
createdAt
updatedAt
```

Profile

```
id
userId
displayName
bio
country
profileImageUrl
createdAt
updatedAt
```

Language

```
id
name
code
```

UserLanguage

Represents a relationship between a user and a language.

```
id
userId
languageId
type
proficiencyLevel
```

`type`:

```
SPEAKS
LEARNING
```

Connection

```
id
requesterId
recipientId
status
createdAt
updatedAt
```

Status:

```
PENDING
ACCEPTED
DECLINED
CANCELLED
```

Conversation

```
id
createdAt
updatedAt
```

ConversationParticipant

```
id
conversationId
userId
joinedAt
```

Message

```
id
conversationId
senderId
type
content
status
createdAt
```

Initial type:

```
TEXT
```

Notification

```
id
userId
type
title
body
readAt
createdAt
```

Block

```
id
blockerId
blockedId
createdAt
```

Report

```
id
reporterId
reportedUserId
messageId
reason
details
status
createdAt
```

---

11. API Requirements

The backend should expose APIs grouped by feature.

Authentication

```
POST /auth/register
POST /auth/login
POST /auth/logout
POST /auth/refresh
POST /auth/forgot-password
POST /auth/reset-password
GET /auth/me
```

Profile

```
GET /users/me
PATCH /users/me
POST /users/me/photo
DELETE /users/me
```

Languages

```
GET /languages
GET /languages/:id
```

User language management:

```
POST /users/me/languages
PATCH /users/me/languages/:id
DELETE /users/me/languages/:id
```

Discovery

```
GET /discover
```

Example parameters:

```
?spokenLanguage=ru
&learningLanguage=en
&country=RU
&page=1
&limit=20
```

User Profiles

```
GET /users/:userId
```

Connections

```
POST /connections/request
GET /connections
GET /connections/requests
PATCH /connections/:id/accept
PATCH /connections/:id/decline
DELETE /connections/:id
```

Conversations

```
GET /conversations
POST /conversations
GET /conversations/:id
```

Messages

```
GET /conversations/:id/messages
POST /conversations/:id/messages
PATCH /messages/:id/read
```

Real-time messaging should additionally use a WebSocket-based connection.

Notifications

```
GET /notifications
PATCH /notifications/:id/read
PATCH /notifications/read-all
```

Safety

```
POST /blocks
DELETE /blocks/:userId

POST /reports
```

---

12. Non-Functional Requirements

Performance

The app should:

• Load core screens quickly

• Paginate discovery results

• Paginate conversations

• Paginate messages

• Avoid downloading unnecessary profile or message data

Scalability

The backend should be stateless where possible.

The system should be capable of scaling individual components such as:

• API servers

• Real-time messaging servers

• Background workers

without requiring a complete rewrite.

Security

Required security practices:

• Password hashing

• Secure authentication tokens

• HTTPS in production

• Authorization checks on every protected resource

• Rate limiting for sensitive endpoints

• Input validation

• Protection against unauthorized profile/message access

• Secure media upload validation

• Environment variables for secrets

A user must never be able to access or modify another user's private data simply by changing an ID in an API request.

Reliability

The system should:

• Handle temporary network failures

• Reconnect real-time communication

• Avoid duplicate messages

• Avoid duplicate connection requests

• Safely handle retries

---

13. Permissions and Privacy

Users should control the information exposed publicly.

For Phase 1, public profile information should be intentionally limited.

Recommended publicly visible information:

• Display name

• Profile photo

• Country

• Bio

• Languages

• Proficiency levels

• Practice goals

Sensitive information such as:

• Email

• Authentication information

• Internal account IDs

must never be publicly exposed.

---

14. Phase 1 Feature Priority

P0 — Must Have for Launch

Accounts

• Registration

• Login

• Logout

• Password reset

• Session management

Onboarding

• Basic profile

• Profile photo

• Spoken languages

• Learning languages

• Proficiency levels

Discovery

• Recommended users

• User profiles

• Basic filters

• Pagination

Connections

• Send request

• Accept request

• Decline request

• Cancel request

Messaging

• One-to-one conversations

• Real-time text messages

• Chat list

• Message history

• Read status

Safety

• Block users

• Report users/messages

Settings

• Edit profile

• Logout

• Delete account

P1 — Strongly Recommended if Time Allows

• Google authentication

• Profile image uploads

• Push notifications

• Search by language/name

• Image messages

• Better match scoring

• Notification center

P2 — Future Phase

• Voice messages

• Voice calls

• Video calls

• AI conversation partner

• AI corrections

• Translation

• Grammar corrections

• Premium subscriptions

• Advanced matching

• Communities

• Group chats

• Public posts/feed

• Language challenges

• Streaks

• Achievements

• Tutor marketplace

---

15. MVP Success Criteria

The first release should successfully allow a new user to complete this scenario:

```
1. Install the application

2. Create an account

3. Complete their profile
```

```
4. Add:
- Languages they speak
- Languages they are learning

5. Discover a relevant language partner

6. View that person's profile

7. Send a connection request

8. Have the request accepted

9. Start a conversation

10. Send and receive messages in real time

11. Continue using the application to find and practice with more people
```

If this experience works smoothly, the Phase 1 product has successfully delivered its primary value.

---

16. Recommended Phase 1 Scope Summary

The Phase 1 application should focus on these **eight core pillars**:

1. **Authentication**
2. **Profile and onboarding**
3. **Language management**
4. **Partner discovery**
5. **Basic matching**
6. **Connection requests**
7. **Real-time one-to-one messaging**
8. **Safety and privacy**

The key principle for the first phase is:

> **Do not try to build every language-learning feature. Build the best possible experience for helping a learner find a relevant person and start practicing through a real conversation.**

Features such as AI tutoring, calls, subscriptions, gamification, and communities can then be built on top of this strong foundation in later phases.

Recommended Additions for Implementation Readiness

The source PRD is strong on product scope and core entities. To make it more useful for design, React Native implementation, QA, and release planning, the following additions are recommended. These are explicitly marked as recommendations rather than requirements from the original PRD.

1. Frontend Architecture & Navigation

Define a screen map, navigation ownership, route guards, authentication/onboarding state, deep-link entry points, loading/error/empty states, and reusable component boundaries.

2. State Management & Data Ownership

Document what belongs in server state versus local UI state, caching rules, invalidation rules, optimistic updates, persistence, and how authentication/session state is restored.

3. API Contract & Error Model

For every endpoint, define request/response schemas, validation errors, authentication failures, pagination format, retry behavior, idempotency expectations, and standard error codes.

4. Real-Time Messaging Contract

Specify WebSocket event names, connection lifecycle, authentication, reconnect/backoff, duplicate message prevention, ordering, acknowledgements, read receipts, and offline message reconciliation.

5. UX State Matrix

For each major screen define loading, empty, populated, error, offline, permission-denied, blocked-user, deleted-user, and retry states.

6. Safety & Moderation Rules

Define what happens immediately after block/report, whether existing chats remain visible, how reported messages are referenced, rate limits for requests/messages, and abuse-prevention behavior.

7. Privacy & Account Lifecycle

Clarify age policy, profile visibility, data deletion behavior, retention, export requirements if applicable, notification consent, and which profile fields are public.

8. Analytics & Product Metrics

Add event names and success metrics for onboarding completion, discovery engagement, connection acceptance, first message, conversation continuation, retention, reports, and notification engagement.

9. Accessibility & Internationalization

Define minimum text scaling support, screen-reader labels, contrast requirements, RTL readiness, translation strategy, and locale/date/time formatting.

10. QA & Release Acceptance Criteria

Turn P0 features into testable acceptance criteria and cover network loss, app restart, duplicate taps, expired sessions, notification deep links, blocked users, and real-time reconnect scenarios.

11. Security & Abuse Controls

Add rate limits, brute-force protection, token storage expectations, secure media handling, input sanitization, authorization tests, and protection against user-ID enumeration.

12. Environment & Configuration

Define development/staging/production environments, API base URLs, feature flags, secrets handling, build configuration, logging policy, and release channels for Android/iOS.

Implementation-Oriented MVP Checklist

■ Authentication: register, login, logout, refresh, password reset, session restore.

■ Onboarding: profile, photo, spoken languages, learning languages, proficiency, practice goals.

■ Discovery: recommended users, search, filters, pagination, relevant language overlap.

■ Profiles: public profile, edit profile, privacy-safe fields.

■ Connections: request, accept, decline, cancel, remove, duplicate-request prevention.

■ Chat: conversation list, message history, text messaging, read state, real-time updates.

■ Notifications: connection request, acceptance, new message, in-app and push behavior.

■ Safety: block, report profile/message, discovery exclusion, messaging restrictions.

■ Settings: edit profile, notification preferences, logout, account deletion, legal pages.

■ Reliability: retry strategy, offline handling, reconnect, duplicate prevention.

■ QA: acceptance criteria for every P0 flow and critical edge case.

■ Analytics: instrument the core funnel from onboarding through first successful conversation.

Important scope note: The original document contains both frontend-facing requirements and backend/API/data-model requirements. This enhanced document preserves the original scope and adds implementation-readiness guidance; the additions are recommendations, not claims that those details already exist in the source.